

Experiment – 1

Solve 8 Puzzle program

Aim:

To develop a Python program to solve the 8-Puzzle problem using the Breadth-First Search (BFS) algorithm and find the shortest sequence of moves from the initial state to the goal state.

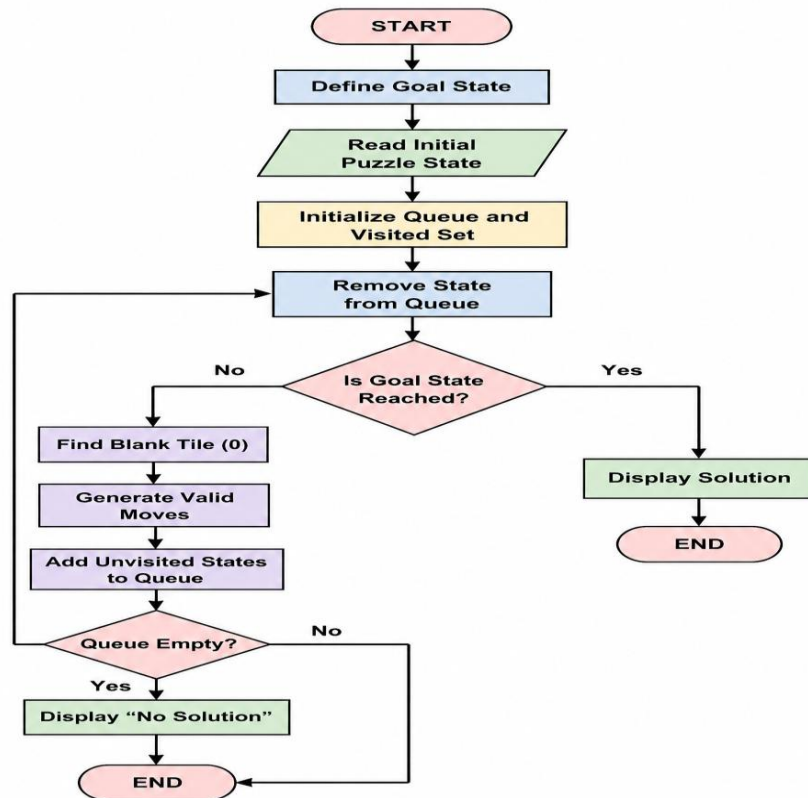
Objective:

- To represent the 8-puzzle problem using a 3×3 matrix.
- To implement the **Breadth-First Search (BFS)** algorithm in Python.
- To find the shortest sequence of moves from the initial state to the goal state.
- To demonstrate state-space search and problem-solving techniques in Artificial Intelligence.

Algorithm:

1. **Start** the program.
2. Define the **goal state** of the 8-puzzle.
3. Read the **initial state** from the user.
4. Create an empty **queue** and insert the initial state into it.
5. Create an empty **visited set** to store explored states.
6. Repeat until the queue is empty
7. If the queue becomes empty and the goal state is not reached, display "**No solution found.**"
8. **End** the program.

Flowchart:



Code:

```
from collections import deque
```

```
goal = [[1,2,3],[4,5,6],[7,8,0]]
```

```
moves = [(-1,0),(1,0),(0,-1),(0,1)]
```

```
def bfs(start):
```

```
    q = deque([(start, [])])
```

```
    visited = set()
```

```
    while q:
```

```
        state, path = q.popleft()
```

```
        if state == goal:
```

```
    return path + [state]
```

```
t = tuple(map(tuple, state))
```

```
if t in visited:
```

```
    continue
```

```
visited.add(t)
```

```
for i in range(3):
```

```
    for j in range(3):
```

```
        if state[i][j] == 0:
```

```
            x, y = i, j
```

```
for dx, dy in moves:
```

```
    nx, ny = x+dx, y+dy
```

```
    if 0 <= nx < 3 and 0 <= ny < 3:
```

```
        s = [r[:] for r in state]
```

```
        s[x][y], s[nx][ny] = s[nx][ny], s[x][y]
```

```
        q.append((s, path+[state]))
```

```
return None
```

```
print("Enter 3x3 puzzle:")
```

```
start = [list(map(int, input().split())) for _ in range(3)]
```

```
ans = bfs(start)
```

```
if ans:
```

```
    for s in ans:
```

```
        for r in s:
```

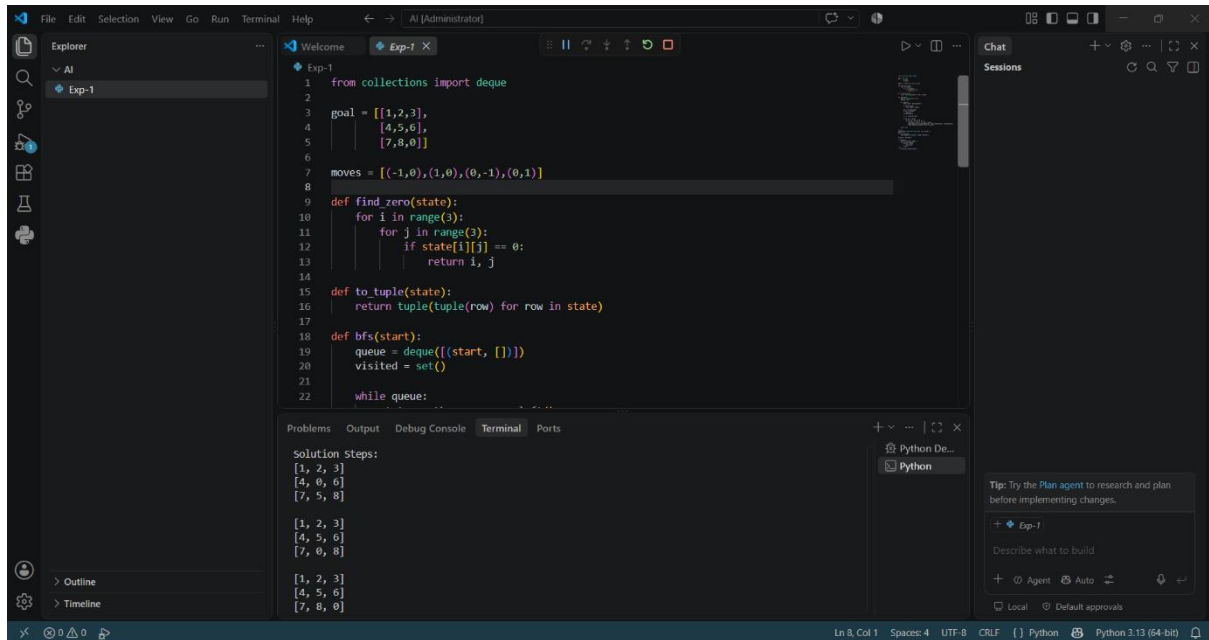
```
            print(r)
```

```
        print()
```

else:

```
print("No solution")
```

Output:



```
1 from collections import deque
2
3 goal = [[1,2,3],
4         [4,5,6],
5         [7,8,0]]
6
7 moves = [(-1,0),(1,0),(0,-1),(0,1)]
8
9 def find_zero(state):
10     for i in range(3):
11         for j in range(3):
12             if state[i][j] == 0:
13                 return i, j
14
15 def to_tuple(state):
16     return tuple(tuple(row) for row in state)
17
18 def bfs(start):
19     queue = deque([(start, [])])
20     visited = set()
21
22     while queue:
```

Solution Steps:

```
[1, 2, 3]
[4, 0, 6]
[7, 5, 8]

[1, 2, 3]
[4, 5, 6]
[7, 0, 8]

[1, 2, 3]
[4, 5, 6]
[7, 8, 0]
```

Result:

The 8-Puzzle problem was successfully solved using the Breadth-First Search (BFS) algorithm, and the shortest path from the initial state to the goal state was obtained.

Conclusion:

The 8-Puzzle problem was successfully solved using the Breadth-First Search (BFS) algorithm, demonstrating its effectiveness in finding the shortest path to the goal state.